

KTB AI 성능 개선 대회

약관 QA 결과기 개선 여정

검색 포화 이후, 원문 충실도와 완결성을 높인 3일간의 개선

개선 목표

검색이 맞아도
답변은 틀릴 수 있었다

공개 10문항에서는 BM25·Dense·Hybrid 모두 정답 조항을 top-4 안에 넣었습니다.
하지만 비공개 30문항은 표현이 달라질 수 있어, 검색 적중만으로는 충분하지 않았습니다.

그래서 목표를 두 갈래로 나눴습니다.

① 공개셋 적중률 유지 ② 표현이 바뀌어도 답변의 핵심 내용·근거·완결성 유지

점수는 검색보다 답변 품질에서 갈렸다

대회 기준은 근거 순위보다 핵심 내용의 누락과 LLM 품질을 더 크게 봅니다.
따라서 검색은 top-4에 정답을 넣는 안전망, 생성은 실제 점수를 만드는 병목
으로 보았습니다.

20점

근거 문서·조항 순위
MRR

30점

핵심 내용 포함률
Key-fact F1

50점

정확성·근거성·완결성·명료성
통합 LLM 평가

검색·생성·선택을 한 흐름으로 묶었다

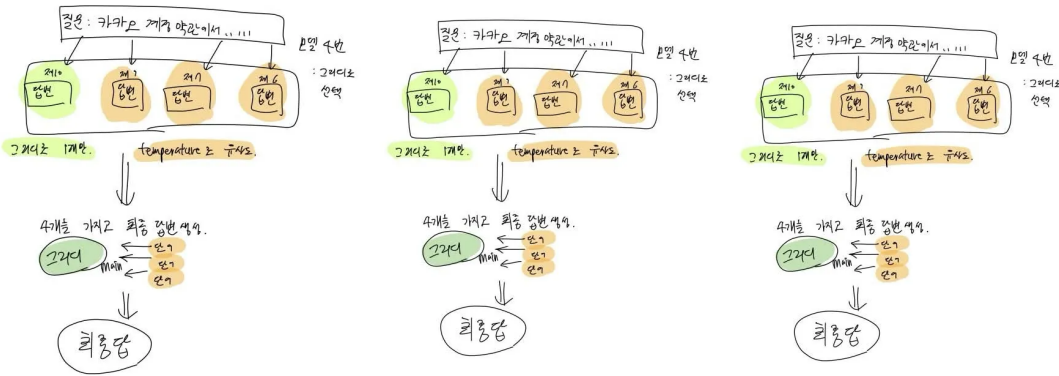
고정 코퍼스 → Hybrid Retrieval → Qwen 생성

약관 4종을 조항 단위 72개 청크로 내장하고,
BM25(어휘 겹침)와 BGE-M3 Dense(의미)를 함께 검색합니다.

질문별 확신도로 가중치를 조절해 RRF로 융합한 뒤 top-4를 선택합니다.

Qwen2.5-7B-Instruct 4bit가 답변 후보를 만들고,
원문 복사율·한국어·잘림·반복 검사를 통과한 후보만 반환합니다.

```
for k in range(1, N_CANDIDATE):
```



공개셋 만점에 멈추지 않고, 질문별 확신도를 실험했다

실험에서 남긴 것

BM25 단독은 표현이 바뀌면 약해지고, Dense 단독은 원문형 질문에 손해가 있었습니다.

그래서 둘을 버리지 않고 확신도 기반 RRF로 결합했습니다.

생성은 그리디 1개 + temperature 0.7 샘플링 3개를 비교합니다.

그리디가 충분히 확실하면 조기 종료하고, 아니면 원문 충실도가 높은 후보로 조건부 교체합니다.

- ✓ BM25 + Dense → 동적 가중치 RRF
- ✓ 조항 단위 청킹 → retrieved 계약 보장
- ✓ Few-shot은 값이 아니라 답변 형식만 학습
- ✓ temperature 0.4 → 0.7: 후보 다양성 확보
- ✓ 7B 4bit + 컨텍스트 상한: 품질·T4 안정성

실패는 구조를 단순하게 만들었고, 다음 병목을 남겼다

최종 공개 연습 결과: 89.054 → 91.035

MRR 1.000 유지 · 핵심 F1 0.7076 → 0.7537 · LLM 95.650 → 96.850

실패

2단계 추출→작문은 핵심 문장을 삭제해 H F1 0.861→0.576.

7B 컨텍스트 무제한은 CUDA OOM.

temperature 0.4는 그리디와 같은 답만 반복.

현재 한계

공개셋 MRR 1.0은 검색 완성을 뜻하지 않음.

로컬 F1 프록시는 공식 LLM 50점을 대체하지 못함.

HTTP p50·p95와 품질을 한 표에서 비교하지 못함.

다음 개선

질문 유형별 원문 span 선택으로 재현율 보강.

비공개형 검증셋을 더 넓혀 과적합 감시.

HTTP p50·p95·성공률과 품질을 함께 기록.

참고 1. 개선 목표와 접근 방향

공개 문항 적중과 비공개 문항 일반화를 동시에 확보

목표	데이터 구성	모델 선택
10개 문항 골드셋에 최대한 적중해 높은 점수를 받는다.	임베딩은 BGE-M3	Qwen 7B 선택
30개 비공개 문항에도 잘 대답해, 공개 10문항에만 과적합되지 않도록 한다.	한국어 특화 임베딩을 사용했다.	Eric의 개인 프로젝트에서 3B를 경험했고, 답변 생성 품질을 높이기 위해 7B로 확장했다.
	벡터 DB 없음	실행 제약
	ChromaDB는 저장(업로드) 후 불러오기가 필요해 대회 규칙 위반이므로 사용하지 않았다.	Drive 마운트가 되지 않는 환경에서 문서를 어떻게 넣을지 함께 해결해야 했다.

참고 2. 실험설계와 근거

모델 메모리, 문서 투입, 검색·컨텍스트 구성을 분리해 판단

양자화 NF4

Qwen 7B는 무거워 양자화가 필수였다. NF4는 정규분포를 잘 따르고, bitsandbytes 안에서 표준 선택이어서 사용했다.

유사도 검색

하이브리드 방식: BM25 + Dense.

컨텍스트 조합

프롬프트를 수정하고, 모델이 주의를 기울이기 쉬운 맨 앞과 맨 뒤를 강조했다.

MD 선택

문서 로딩 시간 없이 바로 넣을 수 있었다. 주제·부제·내용을 간편하게 구성할 수 있어 실행 환경에 맞았다.

참고 3. 개선과정과 기술선택

검색층과 생성층을 각각 실험하고, 실패 원인을 다음 선택에 반영

유사도 검색: 동적 가중치 추가

유사도 검색층을 3계층으로 구성했다.

① BM25로 어휘 겹침 확인

② Dense로 의미 확인

③ 확신도 기반 동적 가중치 → RRF 융합 → top-4 선택

프롬프트

2중 프롬프트를 시도했지만 작업 수행은 이전과 같고 시간만 오래 걸렸다. 그래서 프롬프트 1개를 유지하면서 더 깎았다.

- ✓ Few-shot을 넣은 이유
유도 질문에 답이 이상했다. 예: '본 약관은 세부 지침보다 우선시하나요?'에 바로 '네'라고 답해 근거가 불충실했다.
- ✓ 첫 번째 수정의 문제
'<조항>[예시 약관 제0조 (준칙)] 본 약관에 대해...'를 넣자 답변은 생성됐지만 문항을 못 찾고 retrieved에 제0조가 나왔다.
- ✓ 최종 Few-shot 방향
실제 조문·번호를 넣지 않고 답변 형식만 가르쳤다.
- ✓ Temperature 변경
0.4에서는 그리디로 뽑은 답과 유사한 후보가 계속 나왔다. 그래서 0.7로 올려 후보 다양성을 확보했다.
- ✓ 핵심 교훈
구체적인 조문·값을 few-shot에 넣으면 검색 근거까지 오염될 수 있으므로 형식만 시연한다.

참고 4. 품질과 속도의 균형에 대한 판단

7B의 품질을 유지하면서 후보 생성 비용을 조건부로 줄였다

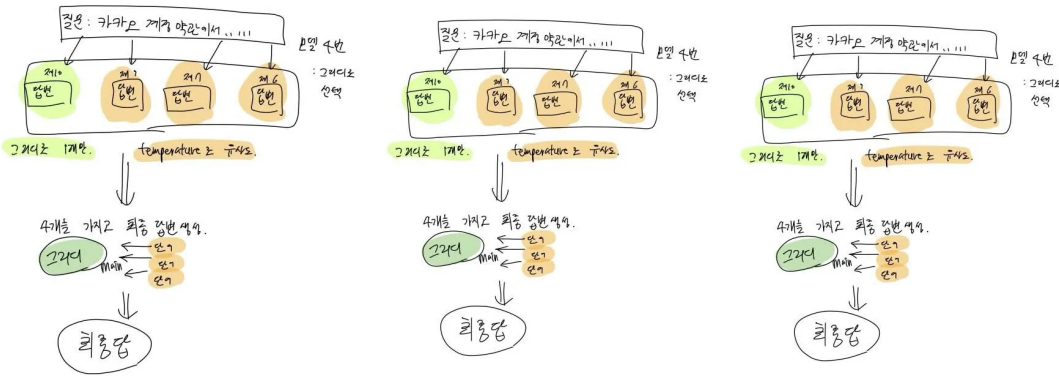
3B는 질문 1개당 10초 이내로 나와 속도는 더 좋았지만, 답변 생성은 7B가 더 우수해서 7B를 선택했다.

후보는 4개를 뽑는다. 그리디 1개 + 샘플링 3개 = 총 4개.

7B를 사용하되 속도를 더 빠르게 하기 위해 early_stopping을 도입했다. 그리디가 0.95 이상이고 어미가 정상일 때는 샘플링 3개를 생략한다. 그보다 낮으면 샘플링 후보를 생성하고 조건부로 교체한다.

실패와 한계, 추가 개선 방향

for k in range(1, N_CANDIDATE):



참고 5. 평가기 전체 구조

결정론적 지표와 LLM 판정을 분리해 비용과 변동성을 줄였다

입력	판정·출력	계산
골드셋 1개 + 익명 답변 파일 5개 + 선택적 약관 원문	셀 5~8	셀 3~4
	Gemini 3.5 Flash가 accuracy·grounding·completeness·clarity 4 축 판정	MRR과 Key-fact F1은 API 호출 없이 계산
셀 1~2	재시도·캐시 후 eval_<팀>.json 저장	골드셋의 문항 id를 그대로 순회해 10개·30개 모두 대 응
설정 → API 키 읽기 → 파일 업로드		
파일명 대신 questions·answers·terms 구조로 판 별	셀 9	후보별 구조 점검
	results만 제출하고 정확한 출력 스키마를 검증	중복·누락·미상 qid 경고, 중복은 첫 번째만 채택

참고 6. 결정론적으로 계산하는 50점

검색 순위와 핵심 내용은 같은 입력이면 같은 점수가 나오도록 만들었다

MRR · 20점

retrieved 최대 4개만 본다. 정답 조항이 처음 나온 순위의 역수다.

1위 1.0 · 2위 0.5 · 3위 0.333 · 4위 0.25 · 미등장 0

문서명은 NFC/NFKC 정규화와 공백 제거, 조항은 ‘3’과 ‘제3조’를 같은 숫자로 읽는다.

실측 근거

조사 제거: F1 0.7561, 공식 0.7537 대비 오차 0.0024

조사 미제거: F1 0.7200, 오차 0.0337

결과기 방향

원문 핵심을 빠짐없이 옮기되 무관한 문장은 줄이는 방향으로 유도했다.

Key-fact F1 · 30점

숫자·영문·한글 토큰을 비교하고, 한국어 조사 제거·불용어 제거로 의미가 같은 표현을 맞춘다.

정밀도와 재현율을 함께 본다. 재현율만 보면 조항 통째 복사가 유리하고, 정밀도만 보면 핵심 단어 하나만 쓰는 답이 유리하기 때문이다.

참고 7. Gemini 판정과 신뢰성 제어

판정은 4축·구조화 출력·재시도로 변동성을 관리한다

판정 4축

accuracy 0.40

grounding 0.25

completeness 0.20

clarity 0.15

축 점수 0~10을 가중합해 0~100으로 환산한다. 정확성에 가장 큰 비중을 둔다.

- ✓ 자료 밖 추론 금지
질문·정답 근거·키팩트·답변만 사용하고 외부 지식으로 보충하지 않는다.
- ✓ grounding 구분
인용을 안 한 것과 틀리게 한 것을 구분한다. 잘못된 조항 인용은 크게 감점한다.
- ✓ 중복 감점 방지
짧다는 이유로 감점하지 않고, 의미가 같으면 인정해 F1과 같은 항목을 두 번 감점하지 않는다.
- ✓ REST·thought 필터
SDK 설치 없이 표준 라이브러리 REST를 사용하고, thought 파트는 JSON 파싱 전에 제거한다.
- ✓ 재시도·캐시
400은 payload 변형, 429·5xx는 4→8→16초 지수 백오프. 문항별 캐시로 중단 후 재개한다.

참고 8. 실행 절차와 출력 계약

후보 답변의 결함과 평가기 실행 실패를 분리해 처리한다

실행 전

- ① 3일차 N QUESTIONS EXPECTED=30
- ② 골드셋 1개 + 익명 답변 5개 업로드
- ③ 예상 Gemini 호출 150회 확인

문항 처리

누락 답변: MRR-F1-Judge를 0점, status는 completed 유지. 빈 답변: API 없이 4축 0점. 우리 API 호출 실패만 partial, 판정 전부 실패만 failed-total:null.

최종 산출

eval_<팀>.json에는 results만 저장. 각 row는 blind_id-total-status 세 필드만 허용. total은 0~100, 다섯 BLIND 후보는 모두 completed인지 검증. 상세 축별 점수·이유는 eval_detail 파일에서 확인.

